VIETNAM NATIONAL UNIVERSITY

HO CHI MINH CITY UNIVERSITY OF SCIENCE

SEMINAR REPORT

Object-Oriented Design Patterns

Course: Programming Systems — CS202
Class: 25A02
Group ID: 53
Seminar topics: Decorator, Facade, and Strategy
Institution: University of Science, VNU-HCM

Ho Chi Minh City, August 2026

Seminar Group Members

The seminar is prepared by Group 53, consisting of two members with complementary responsibilities. Replace the placeholder text below with the group's final information before submission.

Member 1

Name	Từ Hoàng Anh
Student ID	25125005
Primary role	Research and Implementation Lead
Responsibilities	[Example: research pattern theory, develop and test C++ examples, verify class diagrams, and prepare technical demonstrations.]
Contribution	[Enter contribution percentage or summary]

Member 2

Name	Nguyễn Phúc Khánh
Student ID	25125086
Primary role	Documentation and Presentation Lead
Responsibilities	[Example: organize the report, design presentation slides, manage references, prepare the quiz, and coordinate the oral seminar.]
Contribution	[Enter contribution percentage or summary]

Both members should review the complete report and source code, rehearse the presentation, and participate in the question-and-answer session. The final contribution record should agree with the official seminar evaluation form.

Abstract

This report presents three Gang of Four object-oriented design patterns: *Decorator*, *Strategy*, and *Facade*. For each pattern, the report begins with a practical software-design problem, examines a naive implementation, introduces the pattern's intent and participants, and then discusses a C++-oriented implementation. The report also evaluates benefits, limitations, and real-world uses. Together, the patterns demonstrate three different uses of object composition: dynamically adding responsibilities, interchanging algorithms, and simplifying access to a complex subsystem.

Contents

Chapter 1

Introduction

1.1 Background

Object-oriented programming provides encapsulation, inheritance, and polymorphism, but these mechanisms alone do not guarantee maintainable design. As systems grow, recurring problems appear: behavior must be extended without creating many subclasses, algorithms must change without rewriting a context, and clients must interact with complicated subsystems without depending on all their details. Design patterns record reusable structures for addressing such problems.

The Gang of Four catalogue groups patterns into three categories:

- **Creational patterns** control object creation.
- **Structural patterns** organize classes and objects.
- **Behavioral patterns** organize responsibilities and interaction.

Decorator and Facade are structural patterns, while Strategy is behavioral. The seminar emphasizes the principle *favor composition over inheritance* and relates each pattern to the SOLID principles.

1.2 Objectives and scope

The objectives of this seminar are to:

1. recognize the design problems that motivate each pattern;
2. identify the participants and relationships in each pattern;
3. compare naive and pattern-based C++ designs;
4. evaluate benefits, limitations, and appropriate use cases; and
5. distinguish patterns that can appear structurally similar.

The scope is limited to Decorator, Strategy, and Facade. The examples are educational implementations intended to make the object relationships clear; production systems would additionally require error handling, ownership rules, tests, and domain-specific behavior.

1.3 Method

Each pattern follows a problem-first layout: motivating example, naive solution, formal structure, C++ implementation, trade-offs, and applications.

Chapter 2

Decorator Pattern

2.1 Problem: customizable beverages

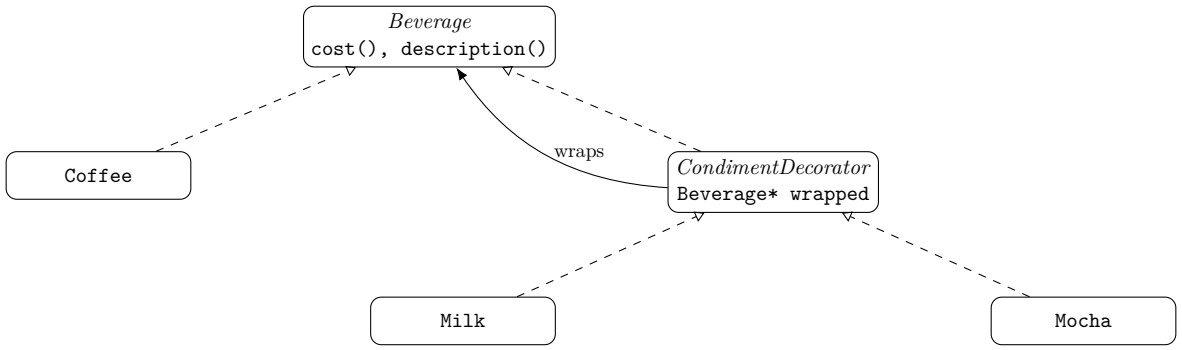
A coffee shop sells a base beverage with optional milk, mocha, sugar, or whipped cream. Every combination changes the description and price. Subclassing for every combination produces classes such as `MilkMochaCoffee` and `DoubleMochaWhipCoffee`. The number of subclasses grows rapidly, and a new condiment affects a large part of the hierarchy.

A second naive approach places Boolean flags in one `Beverage` class. This avoids subclass explosion but mixes pricing rules for all condiments into the base class, weakens cohesion, and makes independent extension difficult.

2.2 Intent and structure

The **Decorator** pattern attaches additional responsibilities to an object dynamically. It provides a flexible alternative to subclassing for extending functionality. Its participants are:

- **Component**: the common interface used by clients;
- **Concrete Component**: the base object being extended;
- **Decorator**: an abstract wrapper that stores another Component;
- **Concrete Decorator**: a wrapper that adds one responsibility.



The decorator implements the same interface as the wrapped object. Therefore, clients can treat a decorated object as an ordinary beverage and wrappers can be stacked at runtime.

2.3 C++ implementation sketch

```cpp
class Beverage {
public:
    virtual ~Beverage() = default;
    virtual std::string description() const = 0;
    virtual double cost() const = 0;
};

class Coffee : public Beverage {
public:
    std::string description() const override { return "Coffee"; }
    double cost() const override { return 2.00; }
};

class CondimentDecorator : public Beverage {
protected:
    std::unique_ptr<Beverage> wrapped;
public:
    explicit CondimentDecorator(std::unique_ptr<Beverage> item)
        : wrapped(std::move(item)) {}
};

class Milk : public CondimentDecorator {
public:
    using CondimentDecorator::CondimentDecorator;
    std::string description() const override {
        return wrapped->description() + ", milk";
    }
    double cost() const override { return wrapped->cost() + 0.40; }
};

std::unique_ptr<Beverage> order = std::make_unique<Coffee>();
order = std::make_unique<Milk>(std::move(order));
```

Ownership is expressed with `std::unique_ptr`, preventing leaks while allowing each decorator to own the next layer. A call on the outermost wrapper delegates inward; results then return through the wrapper chain.

2.4 Evaluation and applications

Decorator supports the Open/Closed Principle, permits runtime composition, and avoids a subclass for every feature combination. However, it can create many small objects, wrapper order can affect behavior, and debugging a deep chain can be difficult. Common examples include Java I/O streams, web middleware, Python function decorators, user-interface widgets, and logging or compression layers.

Chapter 3

Facade Pattern

3.1 Real-world problem: operating a home theater

A home-theater system is simple from the viewer's perspective—start a movie and stop it later—but the operation requires coordination among several independent devices:

- the **amplifier** supplies sound and needs a source and volume;
- the **DVD player** powers on, plays, stops, and powers off;
- the **projector** powers on and selects the DVD input;
- the **screen** must be lowered before viewing and raised afterward;
- the **theater lights** must be dimmed and later restored.

Starting a movie requires the following ordered workflow:

1. dim the lights to 20 percent and lower the screen;
2. turn on the projector and select the DVD input;
3. turn on the DVD player;
4. turn on the amplifier, select DVD, and set the volume to 10; and
5. ask the DVD player to play the requested movie.

Ending the movie reverses the setup: stop playback, power off the amplifier, DVD player, and projector, raise the screen, and restore the lights. Forgetting an operation or changing the order can leave the system in an incorrect state.

3.2 Naive solution without Facade

The naive implementation creates every subsystem in the client and performs the entire workflow directly. The actual seminar source has the following structure:

```
int main() {
    Amplifier amp;
    DvdPlayer dvd;
    Projector projector;
    Screen screen;
    TheaterLights lights;

    // Client manually starts the movie.
    lights.dim(20);
    screen.down();
    projector.on();
    projector.setInput("DVD");
```

```
13      dvd.on();
14      amp.on();
15      amp.setSource("DVD");
16      amp.setVolume(10);
17      dvd.play("The Matrix");
18
19      // Client must also perform the complete shutdown sequence.
20      dvd.stop();
21      amp.off();
22      dvd.off();
23      projector.off();
24      screen.up();
25      lights.on();
26  }
```

This program produces the correct behavior, so the problem is not functional correctness. The problem is the structure of the client and the cost of future change.

3.3 Problems in the naive design

1. **Tight coupling.** The client depends on five concrete subsystem classes and must understand every relevant method.
2. **Complex orchestration.** Correct behavior depends on a long, ordered sequence of calls rather than one operation that expresses intent.
3. **Code duplication.** Every client that starts or stops a movie must repeat the workflow.
4. **Single Responsibility Principle violation.** The client manages its own application logic as well as home-theater coordination.
5. **Maintenance burden.** A changed device API or an additional component such as a subwoofer can affect many client locations.

3.4 Pattern intent and applicability

Facade is a **structural design pattern**. Its intent is to provide a unified interface to a set of interfaces in a subsystem and thereby define a higher-level interface that is easier to use [1]. It is appropriate when:

- clients need a small set of common workflows over a complex subsystem;
- dependencies between clients and implementation classes should be reduced; or
- a layered architecture needs a clear entry point into each layer.

Facade is a convenience boundary, not an access-control mechanism. Subsystem classes may remain public for advanced clients that need fine-grained control. The pattern therefore simplifies the normal path without forcing every possible operation into one interface. It defines new client-facing operations by combining existing subsystem capabilities; it does not add functionality to the subsystem itself.

3.5 General class structure

Figure 3.1 shows the general relationship. The client depends on a Facade, which delegates each high-level operation to the appropriate subsystem objects. An optional Additional Facade can expose a focused subset of operations when placing every workflow in one class would make the

primary Facade too broad. Clients and other facades may use this additional entry point. The subsystems do not need to know that either Facade exists.

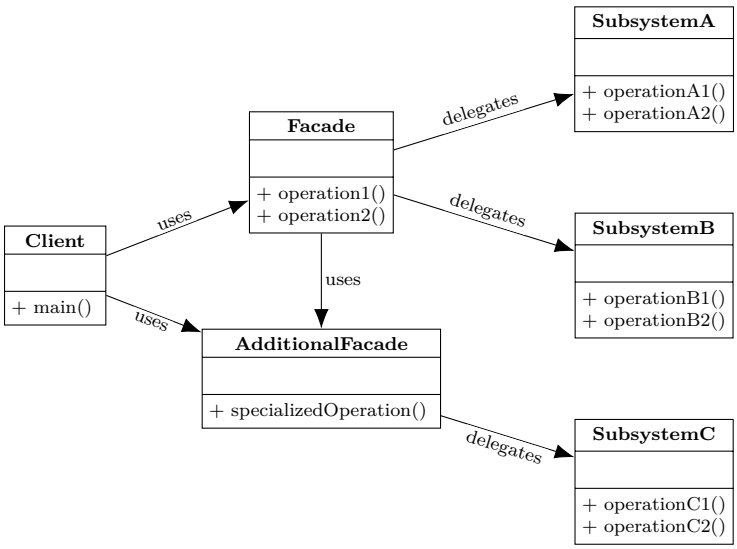


Figure 3.1: General Facade structure with an optional additional Facade.

The primary participants are:

- **Client**: requests a high-level service;
- **Facade**: knows which subsystem objects perform the request and coordinates them;
- **Additional Facade**: provides a smaller, cohesive entry point when the primary Facade would otherwise accumulate unrelated operations; and
- **Subsystem classes**: implement the actual domain operations and can still be used directly when necessary.

3.6 Home-theater class structure

In Figure 3.2, `HomeTheaterFacade` maintains associations to the five subsystem objects and coordinates their operations. Unlike Decorator, the Facade does not implement the same interface as those objects. It offers the high-level operations `watchMovie` and `endMovie`, each of which combines existing device capabilities rather than adding new functionality to the subsystem.

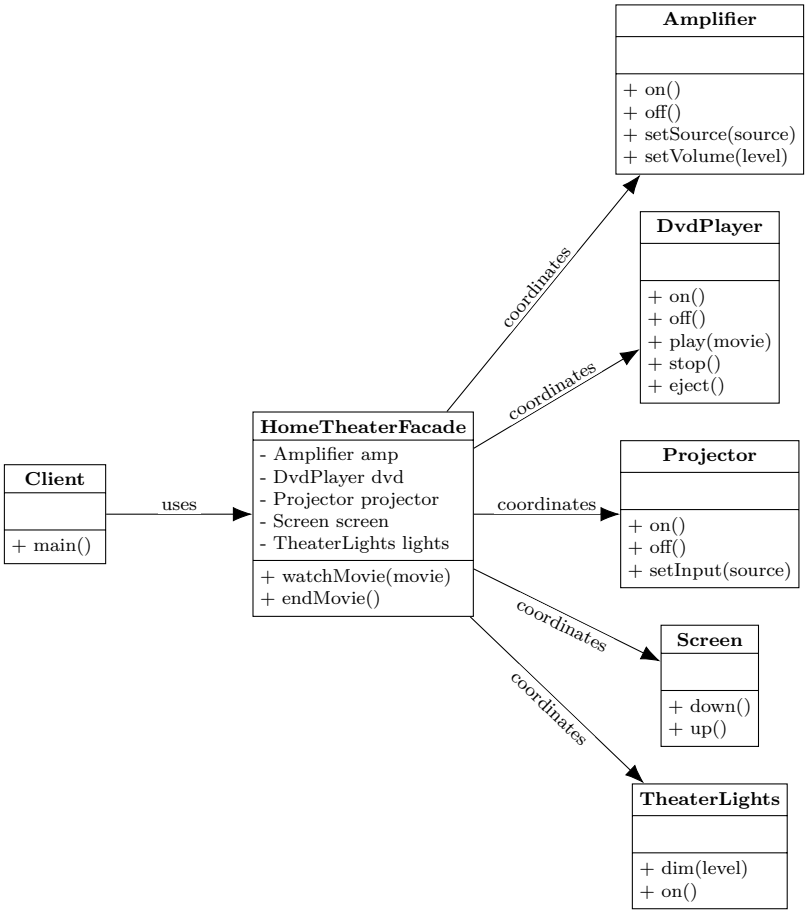


Figure 3.2: Facade structure for the home-theater problem.

3.7 Implementation with Facade

The seminar implementation uses header-only C++11 classes and the standard library. Subsystem objects are created by the client and injected into the Facade through its constructor:

```
class HomeTheaterFacade {
private:
    Amplifier& amp;
    DvdPlayer& dvd;
    Projector& projector;
    Screen& screen;
    TheaterLights& lights;

public:
    HomeTheaterFacade(Amplifier& a, DvdPlayer& d, Projector& p,
                      Screen& s, TheaterLights& l)
        : amp(a), dvd(d), projector(p), screen(s), lights(l) {}

    void watchMovie(const std::string& movie) {
        lights.dim(20);
        screen.down();
        projector.on();
        projector.setInput("DVD");
        dvd.on();
        amp.on();
```

```
        amp.setSource("DVD");
        amp.setVolume(10);
        dvd.play(movie);
    }

    void endMovie() {
        dvd.stop();
        amp.off();
        dvd.off();
        projector.off();
        screen.up();
        lights.on();
    }
};
```

References are suitable for this demonstration for three reasons. They avoid copying objects that conceptually represent hardware, guarantee non-null dependencies, and communicate that the Facade uses but does not own the subsystems. Consequently, the subsystem objects must outlive the Facade.

Client code now communicates intent through two calls:

```
#include "HomeTheaterFacade.h"

int main() {
    Amplifier amp;
    DvdPlayer dvd;
    Projector projector;
    Screen screen;
    TheaterLights lights;

    HomeTheaterFacade theater(amp, dvd, projector, screen, lights);
    theater.watchMovie("The Matrix");
    std::cout << "\n--- Movie is playing ---\n\n";
    theater.endMovie();
}
```

Both the naive and Facade programs compile with `g++ -std=c++11` and produce the same sequence of device output. The improvement is therefore internal design quality: orchestration is defined once and ordinary clients no longer depend on its details.

3.8 Technical considerations

3.8.1 Configuration and flexibility

The demonstration hardcodes a 20-percent light level, volume 10, and DVD input to keep the workflow visible. A production Facade could accept a configuration object while retaining convenient defaults. Advanced clients can also bypass the Facade and call `amp.setVolume` directly with a custom value.

3.8.2 Exception safety

If a device operation fails midway through `watchMovie`, earlier devices may remain active. A production implementation should provide rollback or RAII-based cleanup and must decide what

to do if cleanup itself fails. This is a workflow-transaction concern that the Facade conveniently centralizes.

3.8.3 Open/Closed Principle trade-off

Adding a `Subwoofer` will usually modify the Facade rather than every workflow client, which is a substantial maintenance improvement. However, the current constructor-injection design may also require a change at the application's dependency-assembly point to supply the new object. Strict OCP requirements can be addressed with subsystem interfaces, grouped dependencies, or a specialized Facade, but such flexibility should be justified by expected change rather than added automatically.

3.8.4 Avoiding a god object

A Facade should primarily delegate and coordinate a cohesive set of use cases. It should not absorb unrelated domain state and business logic. When its scope grows, focused interfaces such as `VideoPlaybackFacade` and `AudioCalibrationFacade` are preferable to one universal Facade.

3.9 Advantages and disadvantages

Table 3.1: Trade-offs of the Facade pattern.

Advantages	**Disadvantages**
Isolates ordinary clients from subsystem complexity.	A broad Facade can become a god object.
Reduces coupling and duplicated orchestration.	Adds another delegation layer.
Improves readability through use-case-oriented methods.	A simplified API may omit advanced options.
Localizes common workflow changes.	Changes to injected dependencies may affect the composition root.
Preserves direct subsystem access when needed.	Clients can bypass the Facade and reintroduce coupling.

3.10 Other real-world applications

Web development: API Gateway
A mobile or web client makes one request to a gateway, which handles authentication and coordinates catalog, inventory, ordering, payment, and notification services before returning a unified response.

Mobile development: hardware abstraction
A method such as `Camera.takePhoto` can coordinate sensor access, buffers, image processing, device drivers, and file output behind one operation.

Game development: engine API
High-level operations such as playing a sound or applying force shield game code from rendering, audio, physics, input, and hardware-acceleration details.

Database access: JDBC and ORMs
Operations such as `session.save` hide connection management, SQL generation, database dialects, network protocols, and result processing.

3.11 Relationship to similar patterns

Facade compared with related patterns

Pattern	Primary intent	Key distinction
Adapter	Make incompatible interfaces work together.	Usually translates one object to an existing target interface.
Mediator	Coordinate communication among peer objects.	Colleagues communicate through the mediator; Facade communication is normally client-to-subsystem.
Decorator	Add behavior while preserving an interface.	Decorators share the component interface; a Facade creates a simpler interface.
Proxy	Control access to another object.	A Proxy commonly preserves the subject interface and may enforce access or lifecycle rules.
Singleton	Ensure one instance and a global access point.	A Facade may be a Singleton, but global state harms testing and is not required by Facade.

3.12 Knowledge check

1. Which category contains Facade? (a) Creational (b) Structural (c) Behavioral (d) Concurrency
2. What is its primary purpose? (a) Object creation (b) A simplified subsystem interface (c) One-to-many notification (d) Tree composition
3. In the home-theater example, what changes for an ordinary client? (a) Devices disappear (b) It calls the Facade instead of orchestrating every subsystem (c) Volume becomes automatic (d) Order no longer matters
4. What is a principal risk? (a) Increased inheritance (b) A god-object Facade (c) Forced subsystem rewriting (d) No direct access
5. Which is not a typical Facade example? (a) API Gateway (b) Singleton logger (c) ORM (d) Game-engine API
6. Does Facade prohibit direct subsystem access? (a) Always (b) No (c) Only in C++ (d) Only for private methods
7. Where should a common new subsystem workflow normally be coordinated? (a) Every client (b) The Facade (c) The compiler (d) A decorator

Answer key: 1(b), 2(b), 3(b), 4(b), 5(b), 6(b), 7(b).

3.13 Reviewer focus

Peer reviewers should examine whether the home-theater problem has enough coordination complexity to justify the pattern, whether the naive example makes the coupling and duplication visible, whether both class diagrams express the correct dependency direction, and whether the presentation discusses Facade's limitations as clearly as its benefits. Useful challenges include exception safety, hardcoded configuration, lifetime management, testing with mock subsystems, Facade-versus-Adapter, and the risk of a god object.

Chapter 4

Strategy Pattern

4.1 Real-world problem: navigation route planning

A navigation application must calculate routes between an origin and a destination for several transport modes. Although every mode has the same goal, each requires a distinct algorithm and different data:

- **Driving** prioritizes highways, travel time, traffic conditions, and toll roads.
- **Walking** prioritizes sidewalks, pedestrian crossings, footbridges, and park shortcuts while avoiding roads unsafe for pedestrians.
- **Public transit** evaluates bus timetables, metro lines, walking connections, and transfers.
- **Bicycling** prioritizes bicycle lanes, safe streets, and favorable elevation profiles.

The set of algorithms is expected to grow as the application adds electric scooters, sightseeing routes, accessibility-aware routing, or eco-friendly driving. The design challenge is to support these variations and switch between them at runtime without concentrating every algorithm in one class.

4.1.1 Real-world analogy

Consider a traveler who needs to reach an airport. The destination remains the same, but the traveler can take a bus, hire a taxi, ride a bicycle, or drive a car. Each option is a transportation strategy that makes different trade-offs: the bus may minimize cost, a taxi may prioritize convenience and travel time, a bicycle may suit a short trip with little luggage, and a car may offer greater flexibility. The traveler selects an option according to factors such as budget, time, distance, weather, and luggage. Changing the selected option changes how the journey is performed without changing its overall goal, illustrating how interchangeable strategies provide alternative ways to complete the same task [3].

4.2 Naive solution without Strategy

The naive seminar implementation stores a `TransportMode` value in `Navigator`. Its `buildRoute` method selects an algorithm with a conditional chain:

```
enum class TransportMode {
    DRIVING,
    WALKING,
    PUBLIC_TRANSIT
```

```
};

class Navigator {
private:
    TransportMode mode;

public:
    Navigator(TransportMode initial = TransportMode::DRIVING)
        : mode(initial) {}

    void setMode(TransportMode selected) {
        mode = selected;
    }

    void buildRoute(const std::string& origin,
                    const std::string& destination) {
        std::cout << "Calculating route from " << origin
                  << " to " << destination << "...\n";

        if (mode == TransportMode::DRIVING) {
            std::cout << "Finding the fastest highway route.\n";
        } else if (mode == TransportMode::WALKING) {
            std::cout << "Finding pedestrian paths and shortcuts.\n";
        } else if (mode == TransportMode::PUBLIC_TRANSIT) {
            std::cout << "Checking bus and metro schedules.\n";
        } else {
            std::cout << "Error: unknown transport mode.\n";
        }
    }
};
```

The client changes the enumeration before each request:

```
Navigator navigator(TransportMode::DRIVING);
navigator.buildRoute("Home", "Downtown Office");

navigator.setMode(TransportMode::WALKING);
navigator.buildRoute("Downtown Office", "City Park");

navigator.setMode(TransportMode::PUBLIC_TRANSIT);
navigator.buildRoute("City Park", "Home");
```

This design works for three simple modes, but every routing rule remains part of one monolithic class.

4.3 Problems in the naive design

1. **Open/Closed Principle violation.** Adding `BICYCLING` requires editing the enumeration and the conditional logic in `Navigator`.
2. **Single Responsibility Principle violation.** The context manages route requests and also implements every routing algorithm.
3. **Monolithic growth.** Traffic handling, timetable parsing, elevation analysis, and future route types accumulate in one method or class.
4. **Poor isolated testing.** A routing algorithm cannot be tested as an independent unit; tests must configure and execute the complete Navigator.

5. **Limited runtime composition.** Behavior is selected from a fixed enumeration rather than supplied as a configurable algorithm object.
6. **Merge and maintenance risk.** Multiple developers changing unrelated algorithms must edit the same central file.

4.4 Pattern intent and applicability

Strategy is a **behavioral design pattern**. Its intent is to define a family of algorithms, encapsulate each one, and make them interchangeable so that an algorithm can vary independently from the clients that use it [1].

The pattern is appropriate when:

- a system offers several meaningful variants of one algorithm;
- a class contains large conditional branches that select behavior;
- algorithms should be replaced or configured at runtime;
- algorithm-specific data and dependencies should be isolated; or
- each algorithm needs independent development and testing.

Strategy applies the principle *favor composition over inheritance*. The Navigator context stores a strategy object and delegates routing to it. Unlike a design with a separate Navigator subclass for each mode, one context object can change its behavior without being destroyed and recreated.

4.5 General class structure

Figure 4.1 shows the general participants and dependency direction.

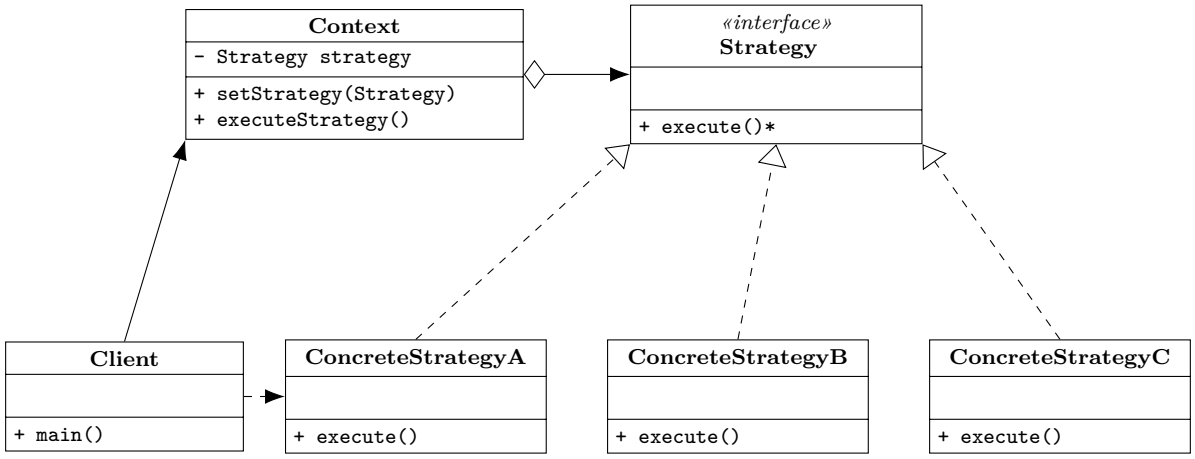


Figure 4.1: General Strategy pattern structure.

- The **Strategy** interface declares the operation shared by all algorithms.
- Each **Concrete Strategy** implements one algorithm.
- The **Context** stores a Strategy reference and delegates the operation instead of implementing the algorithm.
- The **Client** selects a concrete strategy, configures the Context, and may replace the strategy while the program is running.

The Context depends only on the abstraction. Concrete strategies are independent of one another and do not need to know which other algorithms are available.

4.6 Navigation class structure

Figure 4.2 maps the general roles to the seminar example. Navigator is the Context, RouteStrategy is the common interface, and the four transport-mode classes are concrete strategies.

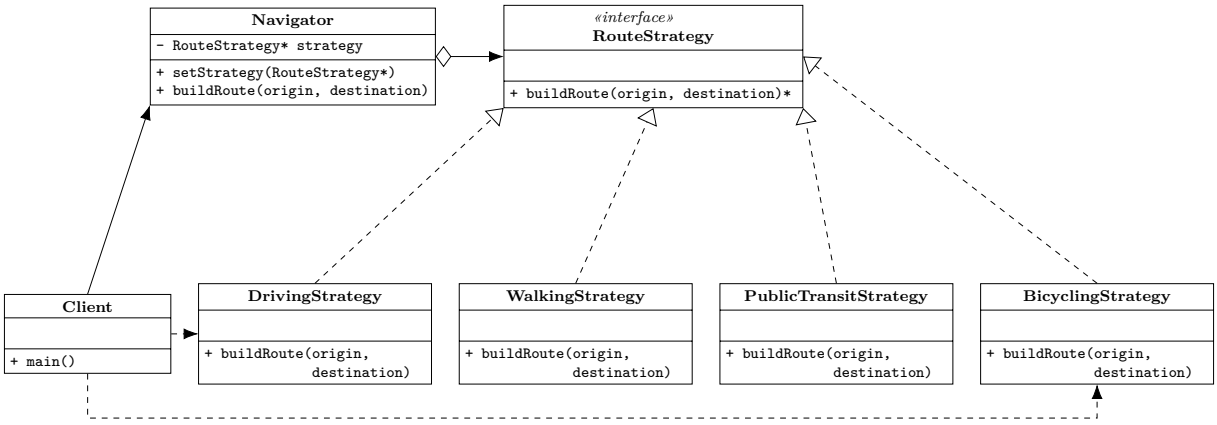


Figure 4.2: Strategy structure for the navigation application.

The hollow-diamond association expresses that Navigator delegates to a strategy object. The demonstration uses a non-owning pointer, so the Client creates both the Context and concrete strategies.

4.7 Implementation with Strategy

4.7.1 Strategy interface and concrete algorithms

The common interface declares a pure virtual route-building operation and a virtual destructor:

```
class RouteStrategy {
public:
    virtual ~RouteStrategy() = default;

    virtual void buildRoute(const std::string& origin,
                            const std::string& destination) const = 0;
};
```

The virtual destructor makes destruction through a base-class pointer safe. The concrete classes implement the same operation with mode-specific behavior:

```
class DrivingStrategy : public RouteStrategy {
public:
    void buildRoute(const std::string& origin,
                    const std::string& destination) const override {
        std::cout << "Mode: DRIVING\n";
        std::cout << "Highway priority, live traffic, and toll roads.\n"
            ;
        std::cout << "Estimated time: 25 mins (18.5 km)\n";
    }
};

class BicyclingStrategy : public RouteStrategy {
public:
```

```
    void buildRoute(const std::string& origin,
                    const std::string& destination) const override {
        std::cout << "Mode: BICYCLING\n";
        std::cout << "Bike lanes and elevation-profile optimization.\n";
        std::cout << "Estimated time: 50 mins (17.1 km)\n";
    }
};
```

The project also supplies walking and public-transit implementations. Adding the new bicycling implementation does not require a change to Navigator or the other strategy classes.

4.7.2 Context

The Context stores the currently selected strategy and forwards route requests:

```
class Navigator {
private:
    const RouteStrategy* strategy;

public:
    explicit Navigator(const RouteStrategy* initial = nullptr)
        : strategy(initial) {}

    void setStrategy(const RouteStrategy* selected) {
        strategy = selected;
    }

    void buildRoute(const std::string& origin,
                    const std::string& destination) const {
        std::cout << "Calculating route from " << origin
                  << " to " << destination << "...\n";

        if (strategy) {
            strategy->buildRoute(origin, destination);
        } else {
            std::cout << "Error: no routing strategy set.\n";
        }
    }
};
```

The null check prevents dereferencing an unset pointer. A production design could instead require a valid strategy in the constructor or inject a Null Object strategy that performs defined fallback behavior.

4.7.3 Client and runtime switching

The actual client creates four stack-allocated strategy objects and switches the Context between them:

```
DrivingStrategy driving;
WalkingStrategy walking;
PublicTransitStrategy transit;
BicyclingStrategy bicycling;

Navigator navigator(&driving);
navigator.buildRoute("Home", "Downtown Office");
```

```
8
9  navigator.setStrategy(&walking);
10 navigator.buildRoute("Downtown Office", "City Park");
11
12 navigator.setStrategy(&transit);
13 navigator.buildRoute("City Park", "Home");
14
15 navigator.setStrategy(&bicycling);
16 navigator.buildRoute("Home", "Riverside Bike Trail");
```

The implementation compiles as C++11 and demonstrates runtime polymorphism. The selected object must outlive the Navigator because the pointer is non-owning.

4.8 Technical considerations

4.8.1 Pointers, ownership, and object slicing

Passing a concrete strategy by pointer or reference preserves virtual dispatch. Passing it by value as a `RouteStrategy` would be invalid because the interface is abstract; more generally, passing polymorphic derived objects by base value causes object slicing. If Navigator should own its strategy, `std::unique_ptr<RouteStrategy>` is a clearer alternative. Shared ownership should be used only when the lifetime genuinely has multiple owners.

4.8.2 Const correctness and demonstration parameters

The declaration `const RouteStrategy*` makes the pointed-to strategy const through this pointer, while the pointer itself remains mutable so that `setStrategy` can replace it. The `buildRoute` member functions are const-qualified, indicating that route calculation does not mutate the strategy object. In the educational source, the concrete strategies receive `origin` and `destination`, but only the Context prints them. A production algorithm would consume these parameters; otherwise their names can be omitted in the definition to avoid unused-parameter compiler warnings.

4.8.3 Client coupling and creation

The Client still needs to select a concrete algorithm. This coupling is localized to configuration rather than spread through the Context and routing workflow. A Factory or dependency-injection configuration can further separate strategy creation from the client that executes routes.

4.8.4 Performance

Virtual dispatch adds one small indirect call and can inhibit inlining. This cost is negligible compared with route search, network access, payment processing, or other substantial algorithms. Highly performance-sensitive inner loops may instead use templates, the Curiously Recurring Template Pattern, or `std::variant` for static or closed-set polymorphism.

4.8.5 Testing

Each concrete strategy can be unit-tested independently. Navigator can be tested with a mock `RouteStrategy` that records the supplied origin and destination, proving that the Context delegates correctly without invoking a real routing service.

4.9 Advantages and disadvantages

Table 4.1: Trade-offs of the Strategy pattern.

Advantages	Disadvantages
Adds algorithms without modifying the Context.	Introduces more classes and objects.
Separates algorithm logic from context logic.	Clients must understand enough to select a strategy.
Supports runtime algorithm replacement.	Adds configuration and virtual-call indirection.
Removes large behavior-selection conditionals.	Can over-engineer simple, stable behavior.
Enables focused unit tests and mocks.	Strategy objects may require careful lifetime management.

4.10 Other real-world applications

Web development: authentication and payment
Authentication middleware can select local-password, Google OAuth, or another provider strategy. Checkout systems can delegate payment to credit-card, PayPal, Apple Pay, or other payment strategies.

Mobile development: layout and caching
Collection views can use linear, grid, or staggered layout strategies. Image libraries can switch among memory-cache, disk-cache, and network-only policies.

Game development: artificial intelligence and physics
A non-player character can replace patrol, attack, and flee strategies as its situation changes. Physics engines can choose collision-detection algorithms according to scene density.

Data processing: sorting and compression
Software can select a sorting, compression, or serialization algorithm based on data characteristics, performance requirements, or output format.

4.11 Relationship to similar patterns

Strategy compared with related patterns

Pattern	Primary intent	Key distinction
State	Represent behavior associated with internal state.	State transitions are often initiated internally; strategies are normally selected externally.
Template Method	Vary steps in an inherited algorithm skeleton.	Template Method uses class inheritance; Strategy uses object composition and runtime delegation.
Command	Encapsulate a request for execution, queuing, logging, or undo.	Strategy represents how an algorithm is performed rather than a request to perform an action.
Decorator	Add responsibilities while preserving an object's interface.	Decorator wraps to add behavior; Strategy replaces the delegated algorithm.
Facade	Simplify access to a complex subsystem.	Facade offers a higher-level interface; Strategy exposes interchangeable implementations of one operation.

4.12 Knowledge check

1. Which category contains Strategy? (a) Creational (b) Structural (c) Behavioral (d) Architectural
2. What is its primary intent? (a) Adapt interfaces (b) Encapsulate interchangeable algorithms (c) Ensure one instance (d) Simplify a subsystem
3. Which principle is most directly supported when a new strategy is added without changing Navigator? (a) Liskov Substitution (b) Open/Closed (c) Interface Segregation (d) Law of Demeter
4. What is the Context's role? (a) Implement every algorithm (b) Store a Strategy and delegate to it (c) Prevent runtime changes (d) Create JSON
5. What is a possible disadvantage? (a) Strategies cannot be tested (b) Clients must select among strategies (c) Context must change for every strategy (d) Runtime swapping is impossible
6. How does Strategy differ from Template Method? (a) Strategy uses composition; Template Method uses inheritance (b) Strategy uses inheritance; Template Method uses composition (c) Both require a Singleton (d) They have identical intent
7. Why use a pointer or reference to the strategy in C++? (a) To force inlining (b) To preserve polymorphism and avoid slicing (c) To create a Singleton (d) To prevent testing

Answer key: 1(c), 2(b), 3(b), 4(b), 5(b), 6(a), 7(b).

4.13 Reviewer focus

Peer reviewers should determine whether the navigation problem genuinely benefits from runtime algorithm replacement, whether the naive implementation clearly exposes conditional growth, and whether the diagrams correctly show composition from Context to Strategy and realization by concrete strategies. Useful challenges include null handling, ownership and lifetime, virtual destructors, object slicing, isolated testing, performance, client knowledge of concrete strategies,

and distinctions from State and Template Method.

Chapter 5

Comparison and Combined Use

Table 5.1: Comparison of the three seminar patterns.

	Decorator	**Strategy**	**Facade**
Category	Structural	Behavioral	Structural
Main question	What behavior can be added?	Which algorithm should run?	How can this subsystem be simpler?
Mechanism	Recursive wrapping	Delegation to a policy object	Delegation to subsystem objects
Interface	Same as wrapped component	Strategy-specific interface	New, simplified interface
Runtime change	Add/remove wrapper layers	Replace the strategy	Usually use fixed workflows
Primary benefit	Flexible extension	Interchangeable algorithms	Reduced client coupling
Typical risk	Too many wrapper layers	Too many strategy classes	God-object facade

The patterns can cooperate in one system. For example, a route-planning service may expose a **Facade** for simple trip planning, use a **Strategy** to select driving or walking algorithms, and apply **Decorators** for caching, metrics, or logging. Pattern choice should follow the design problem; patterns are not goals by themselves.

Chapter 6

Conclusion

Decorator, Strategy, and Facade show how object composition can improve change management in different ways. Decorator extends individual objects without a large inheritance tree. Strategy separates variable algorithms from the context that uses them. Facade shields clients from subsystem complexity through a cohesive high-level interface. Their benefits are strongest when the relevant variation or complexity is genuine; applying them to trivial code can increase indirection without sufficient return. A good design therefore balances flexibility, clarity, ownership, testability, and expected future change.

Appendix A

Submission Checklist

The seminar requirements specify the following submission artifacts:

- AI Usage Declaration and AI Chat Log;
- `Changes.md`, recording changes since the last presentation;
- the presentation in Markdown, PPTX, or Canvas format;
- the presentation in PDF format;
- the report in Word, LaTeX, or Markdown format, including references;
- source code; and
- the seminar member-contribution evaluation.

This LaTeX source and its compiled PDF satisfy the report-source and report-PDF portion of that checklist. The remaining artifacts are maintained as separate submission files.

Appendix B

AI Usage Declaration

Generative AI assistance was used to organize and edit this seminar report, convert the available seminar material into a consistent LaTeX structure, and check formatting and compilation. The group remains responsible for verifying the technical content, source code, citations, and final submission. The corresponding conversation should be exported and submitted as the required AI Chat Log.

Appendix C

AI Chat Log

C.1 Prompts for Writing Reports

The following log records the non-empty user prompts from this report-writing conversation. All displayed timestamps are in Coordinated Universal Time (UTC).

1. **Timestamp:** `2026-08-28 10:01:45.048 UTC`

   I'm doing a Seminar on OOP Design Patterns. The course is called Programming Systems - CS202. The class is 25A02, the group ID is 53. The school is University of Science in HCMC, Vietnam. Edit the `main.tex` file to make it a seminar report, following the report content layout in `Seminar/Requirements.md`.

2. **Timestamp:** `2026-08-28 10:15:03.700 UTC`

   Read and understand all the content inside `Seminar/Facade` folder. Then write and edit the content inside "Facade Pattern" section.

3. **Timestamp:** `2026-08-28 10:20:36.293 UTC`

   In `main.tex`, move the Facade Pattern section before the Strategy Pattern section.

4. **Timestamp:** `2026-08-28 10:33:23.196 UTC`

   In the report PDF, the figure is inserted between the code in the next subsection. Make the diagram appear before the next subsection.

5. **Timestamp:** `2026-08-28 15:41:57.119 UTC`

   One question: Did the "Facade Pattern" section follow each and every step in the file `Requirements.md`?

6. **Timestamp:** `2026-08-28 15:44:06.001 UTC`

   Add a section detailing about the members of the seminar group. There are 2 members, with different roles. Create a template, I'll fill in my own details later.

7. **Timestamp: not exposed**

   Fix all warnings and errors in one pass. Use the available editor and diagnostics tools to inspect issues directly. Use the editor tools to apply changes as needed. Do not include patch blocks, diffs, or code fences in your response. Return a short bullet summary of what you changed.

8. **Timestamp:** `2026-08-28 15:54:46.997 UTC`

Read and understand all the content inside the `Seminar/Strategy` folder. Then write and edit the content in the "Strategy Pattern" section to follow every step in the `Requirements.md` file, like the "Facade Pattern" section.

9. **Timestamp:** `2026-08-29 02:55:00.074 UTC`

   Analyze the content inside "Strategy Pattern" section and the content in this website. Analyze the differences between the two.

10. **Timestamp:** `2026-08-29 03:02:19.581 UTC`

    Fix this problem: "The statement at `main.tex:862` that "the pointer ... is const" is imprecise. `const RouteStrategy*` is a mutable pointer to a const strategy; `setStrategy` proves that the pointer itself changes."

11. **Timestamp:** `2026-08-29 03:12:05.253 UTC`

    In the "Strategy" folder, read and analyze the Mermaid class diagrams in the `REPORT.md` file. Then draw the same UML class diagrams in the Strategy section.

12. **Timestamp:** `2026-08-29 03:15:39.421 UTC`

    Removes ONLY the labels on the arrows in the UML class diagrams.

13. **Timestamp:** `2026-08-29 03:18:18.003 UTC`

    In the "Facade" folder, read and understand the Mermaid class diagrams in the `REPORT.md` file. Then draw the same UML class diagrams in the Facade section.

14. **Timestamp:** `2026-08-29 04:12:31.401 UTC`

    Analyze the content inside "Facade Pattern" section and the content in this website. Analyze the differences between the two.

15. **Timestamp:** `2026-08-29 04:18:28.637 UTC`

    Fix these problems in the "Facade Pattern" section:

    - *The report says the Facade "introduces new high-level operations." `Refactoring.Guru` says Facade introduces a new interface but no new subsystem functionality. These are compatible, but "new operations that combine existing capabilities" would be more precise.*
    - *The UML relationships use the label `wraps`. Coordinates, uses, or delegates to would distinguish Facade more clearly from Decorator.*
    - *General UML diagram does not show `Refactoring.Guru`'s explicit "Additional Facade" participant.*

Appendix D

Member Contribution Record

Group 53 should complete the official member-contribution form and retain a summary of each member's work. Suggested categories are research, source-code implementation, diagrams, report writing, slide preparation, presenting, and question-and-answer preparation. Member names and percentages should be entered before submission.

Bibliography

[1] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.

[2] E. Freeman and E. Robson, *Head First Design Patterns*, 2nd ed. O'Reilly Media, 2020.

[3] Refactoring.Guru, "Strategy Design Pattern". `https://refactoring.guru/design-patterns/strategy`

[4] SourceMaking, "Design Patterns". `https://sourcemaking.com/design_patterns`

[5] ISO/IEC, *ISO International Standard ISO/IEC 14882:2020 — Programming Languages — C++*, 2020.